Measuring Data Exposure in LLM-Integrated Productivity Tools

Marawan Eldeib
Matrikelnummer: 3764796
University of Stuttgart

***Abstract*—LLM-integrated writing assistants run as browser extensions that observe the text a user types or pastes into ordinary web fields. This study measures how much of a confidential document such tools transmit to their own servers *by default* — that is, with no action beyond installing the extension and pasting text, and even when the user has been instructed not to share the document with AI tools. Using an HTTPS man-in-the-middle proxy on an isolated Linux virtual machine, we captured the outbound traffic of two widely used grammar-checking extensions, Grammarly and LanguageTool, as a synthetic "confidential memo" containing twelve unique planted identifiers (including a UUID canary) was pasted into a controlled local page. Across five runs each, both extensions transmitted essentially the entire document (Grammarly 99.0%, LanguageTool 91.9% of characters) and all twelve planted secrets, including the canary, to their servers over encrypted channels; a no-extension baseline transmitted nothing (0%). The result was perfectly reproducible (standard deviation 0.0 percentage points across runs). We frame this not as an exploit but as a gap between default tool behaviour and reasonable user expectation, and we distinguish two behavioural classes — *automatic* transmitters that send on paste versus *on-demand* transmitters that send only on explicit user action. All test data is synthetic and the methodology is fully reproducible.**

I. Introduction

Large-language-model (LLM) integrated productivity tools — grammar checkers, paraphrasers, and "AI assistant" extensions — have become a default part of many users' browsers. They work by observing the content of text fields and sending that content to remote servers for analysis, then returning suggestions. The convenience is real; the privacy cost is largely invisible. A user who has been handed a confidential file and told "do not share this with any AI tool" would not knowingly paste it into a chatbot. But that same user may have a grammar-checking extension enabled in the background, silently watching every field they type or paste into. The question this project asks is deliberately mundane:

> *How does user data exposure differ across LLM-integrated productivity tools during controlled use?*

Concretely: when a confidential document is pasted into an ordinary text field with a writing-assistant extension running by default, how much of that document leaves the machine, and does it include the specifically sensitive parts?

This is not a search for a vulnerability. The tools examined here are doing exactly what they were designed to do. The contribution is a transparent, reproducible *measurement* of that default behaviour and an honest account of what it means for a user's expectations. Our contributions are:

- A controlled, reproducible measurement pipeline (HTTPS interception plus a planted-secret detector) that quantifies how much of a known document reaches a tool's servers, with an unforgeable UUID canary as the headline evidence.
- Empirical results for two independent, widely used grammar-checking extensions (Grammarly and LanguageTool) against a no-extension baseline.
- A distinction between two behavioural classes — *automatic/background* versus *on-demand* transmitters — that explains why not every "AI writing tool" leaks under this threat model.
- A documented account of the tools that were evaluated and excluded, including a cautionary case of a counterfeit extension impersonating a popular product.

II. Background and Threat Model

a) How these tools work.: A writing-assistant extension registers content scripts that attach to editable elements (`<textarea>` and `[contenteditable]`) on visited pages. When text appears in such a field, the extension transmits it — typically over HTTPS or a WebSocket — to its backend, which runs the analysis and returns suggestions. Because the analysis is server-side, the text itself must leave the user's machine for the feature to function at all.

b) Threat model.: We model an ordinary, non-adversarial situation:

> A user receives a confidential file and is explicitly told not to share it with AI tools. They have a writing-assistant extension installed and enabled — the default state for millions of users. In the course of normal work they paste part of the document into a normal text field (an email reply, a web form, a notes box). Without any further action, the extension transmits the content to its servers.

The adversary here is not a criminal; it is the gap between the tool's default behaviour and the user's mental model. The user believes they have withheld the document from AI tools. In fact, merely pasting it into a field an unrelated extension happens to watch has already transmitted it. Our measurement targets precisely this gap: exposure that occurs with *no user action beyond pasting*.

III. Related Work

The privacy risks of browser extensions have been studied for the better part of a decade. Starov and Nikiforakis [27] were among the first to systematically measure the "privacy diffusion" enabled by extensions, showing that many leak identifying information far beyond what their function requires. Weissbacher et al. [30] built Ex-Ray to detect history-leaking extensions, and Aggarwal et al. [1] analysed and detected "spying" extensions that exfiltrate browsing behaviour. Chen and Kapravelos [5] scaled dynamic taint tracking to 181,683 extensions, flagging thousands that leak privacy-sensitive information. These works established that extension data collection is widespread and frequently undisclosed, and they introduced the complementary detection methods — differential string search, content-agnostic network analysis, and taint tracking — that the field still relies on; static data-flow analysis (e.g. DoubleX [12]) later added scale on the code side. This line of work rests on a longer study of the extension security model itself: Carlini et al. [3] evaluated Chrome's permission and isolation architecture and showed that over-broad host permissions undercut its guarantees, Kapravelos et al. [13] dynamically elicited malicious behaviour (including exfiltration) at scale, Somé [25] demonstrated that extensions can expose their privileges to arbitrary web pages, and Pantelaios et al. [19] showed that benign extensions can turn malicious through later updates — which is why a runtime, network-level measurement of what a tool *actually* transmits is more reliable than trusting a single reviewed snapshot.

More recent large-scale studies have moved from browsing metadata to *page content*. Nayak et al. [18] experimentally analysed sensitive-data access by extensions, including access to credentials on login pages, across the Chrome Web Store. Xie et al. [31] built Arcanum, a dynamic taint-tracking system, and found that thousands of extensions — used by an estimated 144 million people — collect sensitive on-page content such as email bodies, social-media activity, and banking information, often without accurate disclosure. Closer to our own design, Bui et al. [2] intercepted the network traffic of roughly 47,000 extensions and compared the observed outbound data flows against each extension's stated privacy practices, finding hundreds whose transmissions contradicted their disclosures. From the tracking side, Dambra et al. [8] measured web tracking from the user's perspective, building on the browser-instrumentation measurement paradigm established by OpenWPM [11].

The most behaviourally similar precedent is "read-before-submit" exfiltration: Senol et al. [24] showed that email addresses and even passwords are sent to third parties *as the user types*, before any form is submitted, and Starov et al. [28] earlier quantified PII leaking from website contact forms. We extend that "transmitted before the user acts" framing from trackers to writing-assistant extensions. Methodologically, our detection step — searching intercepted flows for known sensitive strings — follows a mature network-level PII-measurement lineage: dynamic taint tracking (TaintDroid [10]), ML classification over decrypted mobile flows (ReCon [22]), crowdsourced on-device capture (Lumen [20]), permission-circumvention analysis [21], and longitudinal traffic auditing across app versions [23]. Because outbound payloads can be encoded or obfuscated, we adopt unique planted canaries in the spirit of the black-box *differential* leak detection introduced by AGRIGENTO [6], so that any transmitted fragment is unambiguously attributable to our input.

Closest to the present work, Vekaria et al. [29] audited nine generative-AI browser assistants and, using traffic decryption, showed that several forward full webpage content — including medical and banking details — to their own servers and to third-party trackers. Our study is narrower and complementary: rather than a broad crawl, we perform a deep, controlled, and *fully reproducible* measurement of a specific, common threat model (a confidential document pasted into a field with a grammar checker running by default), using a planted-canary methodology that makes each leak individually and unambiguously attributable. Where prior large-scale work answers "how many extensions leak," we answer "exactly how much of a specific confidential document leaves, and which sensitive tokens, for two representative tools, with what reproducibility." In short, ours is the writing-assistant, canary-proven complement to Vekaria et al.'s assistant-focused audit: the same interception principle, narrowed to a single reproducible threat model and strengthened with unique planted tokens so that every individual leak is provable, not merely observed in aggregate.

Finally, why the transmitted *content* matters is underscored by a parallel line of work on the privacy risks of large language models, which increasingly sit behind these assistants. Carlini et al. [4] showed that language models memorise and can regurgitate verbatim personal data from their training set, and Nasr et al. [17] extended such extraction to aligned, production models, recovering training examples from deployed systems such as ChatGPT; Lukas et al. [14] quantified PII leakage from language models and showed that scrubbing and differential privacy leave residual exposure. Staab et al. [26] showed that models can *infer* sensitive attributes (location, age, income) from ordinary prose; and Mireshghallah et al. [16] showed that LLM-integrated applications routinely disclose information to contextually inappropriate recipients. Text sent to a writing assistant is therefore not inert once it leaves the machine — which is precisely what makes quantifying how much of it leaves, and to whom, worthwhile.

IV. Methodology

A. Study design and conditions

We measure three conditions: two writing-assistant extensions and a control.

- **Grammarly** — the archetypal always-on grammar/style checker and the motivating example for this project.
- **LanguageTool** — an independent grammar checker from a different vendor (LanguageTooler GmbH) that requires no account, making it the cleanest possible replication partner.

- **Baseline** — the identical browser and procedure with *no* extension installed, to capture and subtract ordinary browser background traffic.

Two independent tools exhibiting the same behaviour constitutes within-class replication, which is substantially stronger evidence than a single data point. The scope was reached by evaluating and excluding several other candidates (Section VII); the final design measures *automatic / background* exposure by grammar checkers.

B. Test document and planted secrets

The input is a synthetic "confidential memo" (2,065 characters, 245 words) written to look like an internal compliance document and explicitly marked "DO NOT FORWARD." Embedded in its prose are **twelve unique, fictional identifiers** — names, an email address, a phone number, project and reference codes, and a UUID *canary*, `CANARY-BC267061-67DC-485B-8E51-6F5494765CAB`. None of these strings exists anywhere on the public internet, so any occurrence in captured network traffic is unambiguously attributable to our input. The canary in particular gives a single, unforgeable yes/no signal: if it appears in outbound traffic, the confidential payload provably reached the tool's servers.

C. Interception setup

All captures were performed on an isolated Kali Linux virtual machine so that the man-in-the-middle configuration never touched real browsing. We used mitmproxy [7] (version 12.2.3) as an intercepting HTTPS/WebSocket proxy on `127.0.0.1:8080`. Each tool ran in its own clean Firefox profile with the proxy configured and the mitmproxy certificate authority trusted *only within that profile*, so decryption was scoped to the experiment. Before any data collection, interception was verified per profile (a known HTTPS site must load without a certificate warning and appear in the proxy); this guards against the most dangerous failure mode in such studies, in which a broken interception silently looks identical to a genuine zero-exposure result.

The document was pasted into a bare local HTML page containing a single text field and nothing else — no third-party scripts, analytics, or other domains — served over `http://localhost` so that extension content scripts activate exactly as they would on a real site, but with zero background noise to filter out.

D. Capture procedure

For each run, the document was loaded into the clipboard and pasted with a *real, manual* `Ctrl+V` into the focused field, after which the run recorded for sixty seconds. The manual paste is a deliberate methodological point: a scripted/synthetic paste fills the field visually but does not fire the DOM `input` event the extensions listen for, so the tool never ingests the text and nothing is transmitted — a misleading "clean" zero. A genuine keystroke is what causes the tool to receive the document. Each tool was captured for **five runs** and the baseline for **three**, following an identical, locked protocol so that between-run variance measures reproducibility rather than procedural drift.

E. Metrics

A mitmproxy add-on scored every decrypted outbound request, response, and WebSocket frame; the raw captures were also archived so they can be re-analysed offline. We report, in order of importance:

Planted-secret count (headline).
How many of the twelve identifiers reached the servers, verified individually. The canary makes this unambiguous.

Exposure %.
The fraction of the document's characters whose 20-character sliding window appears anywhere in outbound traffic, after subtracting hosts seen in the baseline. A 20-character window is long enough to avoid false matches from common words and short enough to catch fragmented sends; a per-host concatenated-transcript pass additionally catches content split across multiple frames. Matching is performed over plain, URL-decoded, and JSON-unescaped variants so that encoded transmission is still detected.

Reproducibility.
The standard deviation of exposure % across a tool's runs, labelled High (< 3 pp), Medium (3–10 pp), or Low (> 10 pp).

Traffic visibility.
Reported as *two* separate numbers — the share of intercepted events that used HTTPS, and the count of TLS handshakes that could not be intercepted (e.g. certificate pinning) — never combined into a single figure, because a non-zero failure count means the exposure result is a lower bound.

We additionally report a 95% confidence interval on the mean exposure using a t-distribution ($n-1$ degrees of freedom), and use non-overlapping intervals as informal evidence of a real difference between conditions.

V. Results

A. Headline: both tools transmit the whole document and every secret

Table I and Figure 1 summarise the findings. Both extensions transmitted essentially the entire document and **all twelve planted secrets, including the canary**, to their servers. Grammarly transmitted a median **99.0%** of the document's characters; LanguageTool transmitted **91.9%**. The no-extension baseline transmitted **0.0%** and none of the secrets. All transmission used HTTPS, and there were *zero* uninterceptable TLS handshakes — so the reported figures are complete, not lower bounds.

B. Information types transmitted

Grouping the twelve planted secrets by *type* shows that the leak is not confined to any single class of data. Both tools transmitted every information type present in the document —

TABLE I: Default-configuration exposure per condition. Exposure is the median across runs; secrets are the number of the twelve planted identifiers detected in outbound traffic. "TLS fail" is the count of handshakes that could not be intercepted.

Condition	Runs	Exposure	Std. dev.	Secrets	Canary?	HTTPS	TLS fail
Grammarly	5	**99.0%**	0.0 pp	**12/12**	yes	100%	0
LanguageTool	5	**91.9%**	0.0 pp	**12/12**	yes	100%	0
Baseline	3	0.0%	0.0 pp	0/12	no	—	0

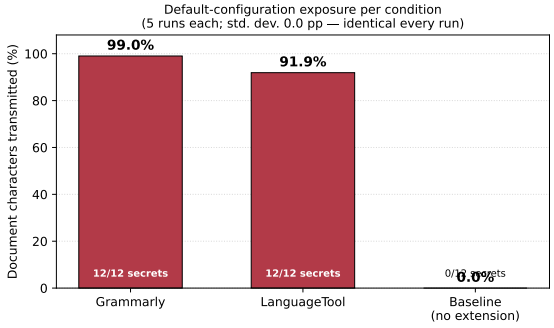

Fig. 1: Share of the confidential document transmitted to each tool's servers by default. Both grammar checkers transmit essentially the whole document and all twelve planted secrets; the no-extension baseline transmits nothing.

personal names, contact details, official identification numbers, financial codes, a legal contract reference, the confidential project codename, and the unique canary — while the no-extension baseline transmitted none of them (Figure 2). Structured identifiers and regulated personal data are therefore exfiltrated just as readily as free-text prose; nothing about being a formatted ID number affords any protection.

C. Perfect reproducibility

Both tools produced *identical* exposure on every run: the standard deviation was 0.0 percentage points. We therefore report this as a *deterministic observation* rather than a statistical estimate. With zero variance the 95% confidence interval has zero width (Grammarly $[99.0, 99.0]$; LanguageTool $[91.9, 91.9]$); we do not read this as a tight sampling estimate or use it to make a significance claim, since a zero-width interval reflects that every repeated measurement was the same, not that a population parameter was estimated precisely. The practical meaning is simply that pasting the document reliably causes the same near-complete transmission every time — consistent with the transmission being a fixed part of the tools' normal operation rather than an occasional or sampled event. Because each tool's per-run value is constant, the two observed values (99.0% and 91.9%) and the baseline (0.0%) are simply different constants, not overlapping estimates.

D. What was transmitted, and how

Grammarly streamed the document to its backend over a WebSocket connection (a median of roughly 6 kB of outbound payload per run across about two tool-related hosts). LanguageTool sent it as an HTTPS `POST` to `api.languagetool.org/v2/check` (roughly 5.6 kB to a single host). In both cases the twelve planted identifiers, including the canary, were recovered from the decrypted outbound traffic. The ∼8 percentage-point gap between the two tools reflects how much surrounding boilerplate each includes rather than any difference in whether the sensitive content leaves: both transmitted *all* of the sensitive tokens.

Viewed over time (Figure 3), the transmission appears as a single outbound burst at the moment the document is pasted, after which the connection falls quiet. The no-extension baseline is instructive: it generates *more* total traffic than either tool run — megabytes of browser and OS background activity — yet none of it carries document content. This is the same reason a baseline condition is required at all: it isolates the environment's own chatter so that document-bearing traffic can be attributed unambiguously to the installed tool rather than to the browser or operating system.

E. Two exposure classes

The excluded-tool investigation (Section VII) surfaced a distinction worth stating as a result. Writing tools fall into two behavioural classes under this threat model: *automatic / background* transmitters, which send the document with no user action beyond pasting (Grammarly and LanguageTool), and *on-demand* transmitters, which send only text the user actively submits (e.g. a paraphraser invoked by selecting text and clicking "rewrite"). Under the study's exact scenario — an extension running in the background while the user merely pastes — the automatic class leaks and the on-demand class does not. Grammar checkers trend toward the automatic class; paraphrasers toward the on-demand class. This is a meaningful distinction for users and policy, not a measurement artefact.

VI. Discussion

The most striking single data point is not the percentage but the canary. A unique token that existed only inside a document marked "DO NOT FORWARD" was found, seconds after an ordinary paste, in traffic bound for two companies' servers — placed there by software the user installed once and forgot. Nothing here is a bug or an exploit: server-side analysis *requires* the text to be uploaded, and both vendors document that they process user text. The finding is about the distance between that design and what a user, told "don't share this with AI," would reasonably believe had happened when they simply pasted a paragraph into an unrelated field.

This distance matters because the exposure is (a) invisible — there is no prompt, no click, no confirmation; (b) complete — essentially the whole document, not a fragment; and

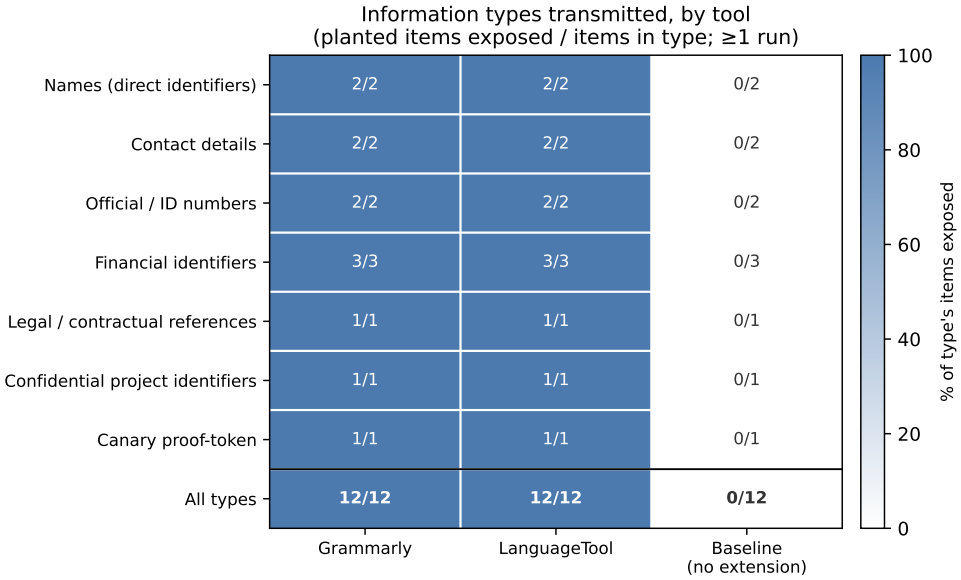

Fig. 2: Information types transmitted to each tool's servers, grouped from the twelve planted secrets (planted items exposed / items in type, over $\geq$1 run). Both writing assistants expose every type; the no-extension baseline exposes none.

(c) deterministic — it happens every time, reliably. For an individual handling regulated data (health, legal, financial) the implication is direct: default-configuration writing assistants are incompatible with a "do not share" instruction unless explicitly disabled, and disabling them is not the default. The two-class distinction gives users a practical lever: automatic grammar checkers are the ones that leak passively, whereas on-demand paraphrasers only transmit what is actively submitted.

VII. Limitations and Excluded Tools

a) Excluded tools (the honest trail).: Not every candidate fit the controlled Firefox method, and recording why is part of the methodology.

- **ProWritingAid** — a genuine extension, but it did not attach to the controlled test field (it sent only small heartbeats, never the document), so it produced no measurable transmission under this protocol.
- **QuillBot** — has no official Firefox extension (Chrome only) and so cannot be tested under a Firefox-based method that must be identical for every tool.
- **Wordtune** — the obvious Firefox listing proved to be a *counterfeit* (a clone under an unrelated publisher, with a scam support link, that echoed text locally and never contacted a server); the genuine tool is published by AI21 Labs but is *on-demand* and therefore does not leak on a background paste. This underscores a practical user lesson: verify an extension's publisher before installing.

b) Other limitations.:

- **Single document.** All runs used one test document; exposure is reported for that document. Because the finding is essentially "all of it, every time," document-to-document variance is unlikely to change the qualitative result, but multi-document measurement would strengthen external validity.
- **Controlled field.** A bare local page isolates the signal but is not a real website; a small number of Gmail/Google-Docs confirmation runs would test representativeness (planned as future work).
- **Tool drift.** These are cloud products that auto-update, and prior longitudinal work shows extension/app privacy behaviour shifts across versions [23]. The *pipeline* reproduces exactly; the specific percentages reflect tool behaviour at capture time (each run is timestamped).
- **Scope of interception.** Certificate pinning can hide traffic; here the count of un-interceptable handshakes was zero, so this did not affect the result, but in general the exposure figure is a lower bound. TLS interception can itself alter or weaken the connection it observes [9]; our proxy CA is trusted only in the isolated test profile, and we report the raw exposure without relying on any modification of the traffic.
- **Single environment.** All runs used one operating system and one browser (Firefox on a controlled Linux VM). Behaviour could differ on other browsers or on the tools' desktop applications; extending to the desktop and other browsers is planned as future work.
- **Manual trigger timing.** The paste that triggers transmission was performed by hand, so the absolute time of the transmission varies between runs. This affects only *when* the spike occurs, not *whether* it does, nor the measured exposure.
- **Unequal run counts.** Five runs per tool and three baseline runs. Because every run of a given condition was identical (zero variance), additional runs would not

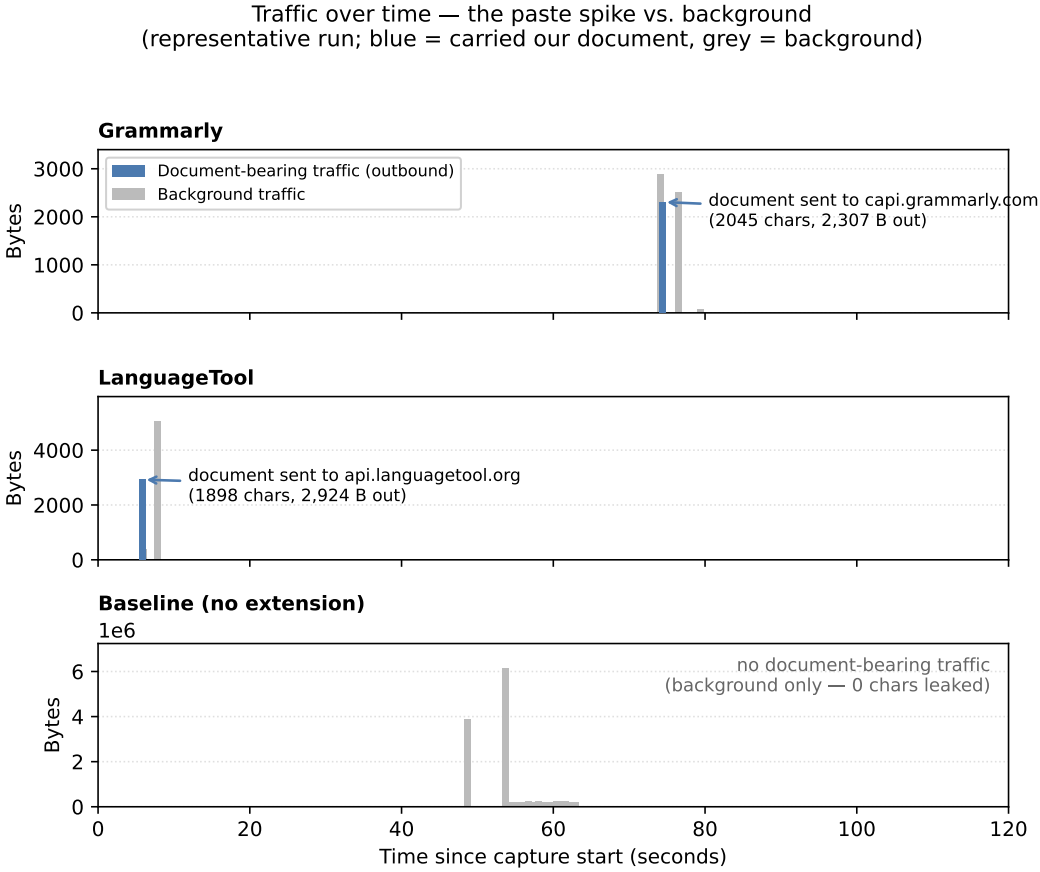

Fig. 3: Traffic over time for a representative run. Both writing assistants transmit the document in a single outbound burst at the moment of paste (blue); the no-extension baseline produces only background traffic (grey) and never transmits document content. Paste timing differs between runs because it was triggered manually — the salient feature is the shape (a single document-bearing spike vs. none), not the absolute time.

change the reported values, but the asymmetry is noted for completeness.

- **Detection method.** Sensitive tokens are detected by case-insensitive substring match and document coverage by a 20-character sliding window. Because the planted identifiers are long and synthetically unique, the risk of a coincidental (false-positive) match in unrelated traffic is negligible; the 20-character window likewise avoids matches on common short strings.

VIII. Ethics and Responsible Disclosure

All test data is synthetic: no real person, organisation, or system is referenced, and no real personal data was captured or transmitted. The study measures default-configuration behaviour of legitimately installed software on the researcher's own machine; it does not attack the vendors' systems or other users. Consistent with responsible-disclosure norms, findings are intended to be shared with the affected vendors, with a reasonable window to respond, before any public release of the repository.

IX. Conclusion and Future Work

Two independent, widely used grammar-checking browser extensions each silently transmitted essentially an entire confidential document — and every one of twelve planted secrets, including a unique canary — to their servers, automatically, on a single paste, with perfect reproducibility, while a no-extension baseline transmitted nothing. The tools are behaving as designed; the result documents how far that design sits from the expectations of a user told not to share a document with AI tools, and it distinguishes the automatic transmitters that leak passively from on-demand tools that do not. Future work includes a small set of real-field (Gmail, Google Docs) confirmation runs, extension to additional tools and multiple documents, and measurement of whether disabling or configuring the tools changes the default outcome. It also includes moving beyond the browser to the tools' *desktop and system-level* clients — which requires a system-wide proxy and, where certificate pinning is used, dynamic instrumentation (e.g. Frida) to observe the traffic — and testing background/idle behaviour and requested device permissions, in line with the broadened scope agreed with the supervisor. Beyond the

twelve planted secrets, an automated PII detector such as Microsoft Presidio [15] can be run over the decrypted traffic to discover and classify *unplanted* personal data, turning the raw captures into a per-category count of exposed information.

References

[1] Anupama Aggarwal, Bimal Viswanath, Liang Zhang, Saravana Kumar, Ayush Shah, and Ponnurangam Kumaraguru. I spy with my little eye: Analysis and detection of spying browser extensions. In *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 47–61, 2018.

[2] Duc Bui, Brian Tang, and Kang G. Shin. Detection of inconsistencies in privacy practices of browser extensions. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (S&P)*, pages 2780–2798, 2023. doi: 10.1109/SP46215.2023.10179338.

[3] Nicholas Carlini, Adrienne Porter Felt, and David Wagner. An evaluation of the Google Chrome extension security architecture. In *Proceedings of the 21st USENIX Security Symposium*, 2012.

[4] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *Proceedings of the 30th USENIX Security Symposium*, 2021.

[5] Quan Chen and Alexandros Kapravelos. Mystique: Uncovering information leakage from browser extensions. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1687–1700, 2018.

[6] Andrea Continella, Yanick Fratantonio, Martina Lindorfer, Alessandro Puccetti, Ali Zand, Christopher Kruegel, and Giovanni Vigna. Obfuscation-resilient privacy leak detection for mobile apps through differential analysis. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017. doi: 10.14722/ndss.2017.23465.

[7] Aldo Cortesi, Maximilian Hils, Thomas Kriechbaumer, and mitmproxy contributors. Mitmproxy: A free and open source interactive HTTPS proxy, 2010–2026.

[8] Savino Dambra et al. When sally met trackers: Web tracking from the users' perspective. In *Proceedings of the 31st USENIX Security Symposium*, 2022.

[9] Zakir Durumeric, Zane Ma, Drew Springall, Richard Barnes, Nick Sullivan, Elie Bursztein, Michael Bailey, J. Alex Halderman, and Vern Paxson. The security impact of HTTPS interception. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017.

[10] William Enck, Peter Gilbert, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2010.

[11] Steven Englehardt and Arvind Narayanan. Online tracking: A 1-million-site measurement and analysis. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016. doi: 10.1145/2976749.2978313.

[12] Aurore Fass, Dolière Francis Somé, Michael Backes, and Ben Stock. DoubleX: Statically detecting vulnerable data flows in browser extensions at scale. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021.

[13] Alexandros Kapravelos, Chris Grier, Neha Chachra, Christopher Kruegel, Giovanni Vigna, and Vern Paxson. Hulk: Eliciting malicious behavior in browser extensions. In *Proceedings of the 23rd USENIX Security Symposium*, 2014.

[14] Nils Lukas, Ahmed Salem, Robert Sim, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Analyzing leakage of personally identifiable information in language models. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (S&P)*, pages 346–363, 2023.

[15] Microsoft. Presidio: Data protection and de-identification SDK. https://github.com/microsoft/presidio, 2018. Open-source software; docs at https://microsoft.github.io/presidio/.

[16] Niloofar Mireshghallah, Hyunwoo Kim, Xuhui Zhou, Yulia Tsvetkov, Maarten Sap, Reza Shokri, and Yejin Choi. Can LLMs keep a secret? Testing privacy implications of language models via contextual integrity theory. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.17884.

[17] Milad Nasr, Javier Rando, Nicholas Carlini, Jonathan Hayase, Matthew Jagielski, A. Feder Cooper, Daphne Ippolito, Christopher A. Choquette-Choo, Florian Tramèr, and Katherine Lee. Scalable extraction of training data from aligned, production language models. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. arXiv:2311.17035.

[18] Asmit Nayak, Rishabh Khandelwal, Earlence Fernandes, and Kassem Fawaz. Experimental security analysis of sensitive data access by browser extensions. In *Proceedings of the ACM Web Conference (WWW)*, 2024.

[19] Nikolaos Pantelaios, Nick Nikiforakis, and Alexandros Kapravelos. You've changed: Detecting malicious browser extensions through their update deltas. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2020. doi: 10.1145/3372297.3423343.

[20] Abbas Razaghpanah, Rishab Nithyanand, Narseo Vallina-Rodriguez, Srikanth Sundaresan, Mark Allman, Christian Kreibich, and Phillipa Gill. Apps, trackers, privacy, and regulators: A global study of the mobile tracking ecosystem. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018.

[21] Joel Reardon, Álvaro Feal, Primal Wijesekera, Amit Elazari Bar On, Narseo Vallina-Rodriguez, and Serge Egelman. 50 ways to leak your data: An exploration of apps' circumvention of the Android permissions system. In *Proceedings of the 28th USENIX Security Symposium*, pages 603–620, 2019.

[22] Jingjing Ren, Ashwin Rao, Martina Lindorfer, Arnaud Legout, and David Choffnes. ReCon: Revealing and controlling PII leaks in mobile network traffic. In *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2016. doi: 10.1145/2906388.2906392.

[23] Jingjing Ren, Martina Lindorfer, Daniel J. Dubois, Ashwin Rao, David Choffnes, and Narseo Vallina-Rodriguez. Bug fixes, improvements, and privacy leaks: A longitudinal study of PII leaks across Android app versions. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018. doi: 10.14722/ndss.2018.23143.

[24] Asuman Senol, Gunes Acar, Mathias Humbert, and Frederik Zuiderveen Borgesius. Leaky forms: A study of email and password exfiltration before form submission. In *Proceedings of the 31st USENIX Security Symposium*, pages 1813–1830, 2022.

[25] Dolière Francis Somé. EmPoWeb: Empowering web applications with browser extensions. In *Proceedings of the 2019 IEEE Symposium on Security and Privacy (S&P)*, 2019. doi: 10.1109/SP.2019.00058.

[26] Robin Staab, Mark Vero, Mislav Balunović, and Martin Vechev. Beyond memorization: Violating privacy via inference with large language models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.07298.

[27] Oleksii Starov and Nick Nikiforakis. Extended tracking powers: Measuring the privacy diffusion enabled by browser extensions. In *Proceedings of the 26th International Conference on World Wide Web (WWW)*, pages 1481–1490, 2017.

[28] Oleksii Starov, Phillipa Gill, and Nick Nikiforakis. Are you sure you want to contact us? Quantifying the leakage of PII via website contact forms. In *Proceedings on Privacy Enhancing Technologies (PoPETs)*, pages 20–33, 2016. doi: 10.1515/popets-2015-0028.

[29] Yash Vekaria, Aurelio Canino, Jonathan Levitsky, Alex Ciechonski, Patricia Callejo, Anna Maria Mandalari, and Zubair Shafiq. Big help or big brother? Auditing tracking, profiling and personalization in generative AI assistants. In *Proceedings of the 34th USENIX Security Symposium*, 2025.

[30] Michael Weissbacher, Enrico Mariconti, Guillermo Suarez-Tangil, Gianluca Stringhini, William Robertson, and Engin Kirda. Ex-ray: Detection of history-leaking browser extensions. In *Proceedings of the 33rd Annual Computer Security Applications Conference (ACSAC)*, pages 590–602, 2017.

[31] Qinge Xie, Manoj Vignesh Kasi Murali, Paul Pearce, and Frank Li. Arcanum: Detecting and evaluating the privacy risks of browser extensions on web pages and web content. In *Proceedings of the 33rd USENIX Security Symposium*, 2024.